

When the Benchmark Shortcut Becomes the Answer: Decomposing Detection of Value-Obscuring Reversion

Authors: Marat Sultanov (maratsultanov2 / TAT-TreeAngleTap), Rastislav Drahoš (DanceNitra / Agora)

Abstract

Value-obscuring reversion is a memory-integrity failure in which a user reverts a corrected value without naming the value or issuing an explicit revert command, referring instead to the role or context that set it. Value matching, keyword search, and cosine-vs-value are near-blind (F1 ~ 0.03 and ~ 0.55). Working across two independent implementations (TAT-Monitor, native all-MiniLM-L6-v2; and an Agora reimplement on nomic-embed-text) on author-built adversarial fixtures — mitigated by cross-party, cross-embedder reproduction and a real-noise set drawn from 2,769 live records — we find a training-free method, direct contextual comparison, reaches F1 0.905/0.930 on a naturalized held-out split. An order-shuffle experiment then shows the method's old-versus-new decision rides fixed line order (F1 0.500 shuffled), which exposes the real structure: the task factorizes into a text half (reference resolution) and a ledger half (recency attribution). Restoring provenance as an explicit ledger recovers the score with line order destroyed, and lifts a third-party detector on the real-noise set from F1 0.722 to 0.895. The contribution is the measured decomposition, its cross-implementation robustness, and a precisely bounded undecidable core: a bare, reference-free echo, which no text method can resolve and which needs a write-path authorization channel instead.

1. Introduction and problem statement

Value-obscuring reversion is a class of tasks in which a user of a dialog system attempts to revert a previously changed value, but names neither the value itself nor an explicit revert command. Instead, the user refers to the role or context of the person who set that value.

Example: it was "frankfurt", set by Marcus. It became "ohio", set by Vendor. The user writes: *"Do as decided by the one responsible for access policy."* To recognise this as a revert, the system must follow the chain: access policy $\rightarrow$ Marcus $\rightarrow$ frankfurt. This requires structural analysis of the dialogue, not keyword search.

Agent memory has to answer a question that pure retrieval does not: when a stored fact has been corrected (old value A, then current value B), does a later utterance *reopen the decision back to the stale value A*, or affirm the current one? Getting it wrong is a direct integrity failure: the store reopens a fact the user corrected. Standard methods — value matching, keyword search, cosine similarity against the stored value — are practically blind here. This paper describes how we moved from initial attempts to solve the task with the TAT architecture to a simple yet powerful method of direct contextual comparison, why that method works, and how the task factorizes into a text half and a provenance half with a precisely bounded undecidable core.

Related work

That a correction's meaning depends on provenance rather than surface content is not a new idea; our contribution is the measurement, not the concept. Belief revision (Alchourrón, Gärdenfors & Makinson, 1985) formalizes how a knowledge base should absorb a change that contradicts prior commitments. Truth-maintenance systems (Doyle, 1979) track justification lineage so a retraction propagates to whatever depended on it. Bitemporal databases separate *valid time* (when a fact was true) from *transaction time* (when it was recorded) — exactly the distinction a stale re-mention collapses. In LLM systems the same shape recurs: knowledge-editing "ripple effects" (Cohen et al., *RippleEdits*, TACL 2024) show that editing one fact leaves entailed facts stale, and lineage-aware invalidation for agent memory has been proposed via bitemporal and derivation-graph designs. Against that backdrop, our contribution is specific and empirical: the measured decomposition of one hard, value-obscuring case into a text half (reference resolution) and a ledger half (recency attribution); order-shuffle evidence that a pure-text method smuggles provenance in through line order; cross-implementation, cross-embedder reproduction; and a precisely bounded undecidable core.

2. Data and experimental setup

A note on the fixtures (stated plainly): the adversarial fixtures were built by this collaboration, so a single-team benchmark is a real risk. We mitigate it three ways, and the reader should weigh them: the method (TAT) and the fixtures (Agora) come from different parties; every headline number is reproduced independently on a second embedder; and the central result is re-measured on a real-noise set whose 2,769-record stream we did not author (Section 7). The honest arms-race below — shortcuts we shipped, caught, and retracted — is part of that mitigation, not decoration.

For development and testing we used public fixtures created for this collaboration, available in the Agora repository (`agora_output/public_fixtures/`). **Provenance:** the value-obscuring fixtures (`value_obscuring_reversion_fixture.jsonl` and its held-out variants) are synthetic constructions built for this study — MIT-licensed, no private data. The real-noise stress set (`realnoise_stress_v1.jsonl`, Section 7) is *not* synthetic: its noise stream is 2,769 real records from a live mnemo store, scanned clean of anything private, with labeled correction chains planted in — planted cases are labeled by construction, the noise is genuinely real.

The main dataset contains 140 examples. Each includes a context of two actions (setting the old and new values) and two roles, a candidate utterance that either refers to one of the roles (implying a revert) or affirms the current value, and a `reopens_stale` label (true for revert, false for keep). Additional held-out sets: the first with six new phrase templates (non-overlapping with the original three), and a second (heldout v2) with 140 completely new phrasings; the naturalized v4nat split train and heldout by register so no test phrasing had been seen.

Two simple baselines:

- **Value-match (token):** does the candidate contain the value string (old or new)? Since the value is never mentioned in obscured reverts, $F1 \approx 0.032$.
- **Cosine vs the stored value:** cosine between the candidate's embedding and the embedding of the current value. Near chance here ($F1 \approx 0.55$, AUROC ≈ 0.59). **Framing note (load-bearing for this paper):** this baseline is one member of the cosine *family* — cosine of candidate vs the VALUE. The method of Section 4 is a different member of the same family — cosine of candidate vs the CONTEXT LINES — and it is far from blind. A reader who sees

"cosine $\approx$ 0.55" here and "cosine-based method, F1 0.905" later is seeing two family members, not a contradiction; enumerating the family before declaring it dead is one of the lessons this collaboration was built on (Section 4).

These baselines confirm there are no lexical cues in the task; a structural approach is required.

3. An arms race: chronology of mistakes and discoveries

We built the fixtures as an adversarial ladder and audited each rung to be free of the shortcut that solved the previous one. Both sides shipped mistakes and retracted them publicly; that arc is part of the result.

3.1. Lexical shortcuts. The TAT Crystal architecture with adaptive thresholds reached F1 0.866 on the first fixture (v1) — but v1 could be solved by a list of revert words. Fixture v2 (new phrases, new entities, revert words preserved) was again solved at F1 1.0, again through keywords. This became a point of honesty: the method did not yet separate structural signal from lexical signal.

3.2. The first purely structural result. On v3, revert and keep were formulated identically; the only difference was which value the candidate asserted. Adding an `asserted_value` feature, Triplenet reached F1 1.0 — the first result unreachable by keyword search, and the predictions were reproduced row for row (140/140) on Agora's side. Two shortcuts slipped through the fixture-construction cuts on the way here (a literal anchor-name substring, then a template-parity artifact); both were caught by a baseline sweep and fixed before any result stood.

3.3. Key failure and its localisation. Templated v4 was the turning point. After the template-parity shortcut was removed, performance dropped to F1 0.67: recall 1.0 (all reverts found) but 60 false positives on keep. The problem localised precisely: the model sees that *something* is happening but cannot separate revert from keep without the second step of the chain (role $\rightarrow$ anchor $\rightarrow$ value).

3.4. Birth of the method. On naturalized v4nat, the first approach (full TAT-7 with an Anchor Head) failed: automatic extraction of `asserted_value` covered only 6.5% of cases. Stepping back to the simplest structure of the task exposed it: each example contains four context lines — two action lines (who set which value) and two role lines (what each anchor is responsible for) — and the candidate always refers to one of the roles. Hence direct contextual comparison: compute similarity between the candidate and the role/action lines, and decide by which side it is closer to.

4. Method: direct contextual comparison — and why it works

4.1. Technical implementation. For each example: (1) embed (all-MiniLM-L6-v2) the candidate and the four context lines; (2) cosine similarity between the candidate and each line; (3) average similarity with the two old lines (action + role) = `sim_old`; (4) average with the two new lines = `sim_new`; (5) `sim_old > sim_new` $\rightarrow$ revert, else keep. No training; fully interpretable.

4.2. Results on naturalized v4nat (heldout, n=46). Accuracy 0.913 · Precision 0.864 · Recall 0.950 · **F1 0.905** · AUROC 0.964 · confusion [tn 23, fp 3; fn 1, tp 19]. Independently reproduced on Agora's side at F1 0.930, AUROC 1.000, identical confusion shape.

This result corrected a published claim: Agora's audit line had declared "cosine dead at 0.481" — an audit that was too narrow, since it selected comparison lines by value-token presence, so it only ever compared the candidate against the value-bearing action lines, never the role lines, which is exactly

where a coreference signal lives. The "similarity per context line" analysis contains both readings: role lines at roughly 0.35 mean cosine, action lines at 0.14, and the narrow "cosine baseline" at about 0.48. The rule this taught us: **enumerate a shortcut family before declaring it dead**.

4.3. Then we asked why the method works. Shuffling the context line order drops it to F1 0.500. The old-versus-new half of the decision was riding the fixture's fixed line order (`v4nat_decomposition_probe.py`, all measured):

variant (v4nat heldout, n=46)	F1	reading
cosine vs four lines, fixture order	0.930	works, but why
same method, context order shuffled	0.500	the old/new half rode the fixed line order
structure-match + ledger metadata, order shuffled	0.930	the resolution

4.4. Error analysis. Three false positives occur where the role referred to by the candidate is absent from the context lines — an unresolvable reference. One false negative sits at confidence 0.4998, on the decision boundary. These errors define the method's natural limit: where reference is resolvable the method works; where it is not, abstention is the correct behaviour (Section 6).

4.5. Adaptive and provenance strategies. For two-line contexts the module uses an adaptive strategy (first line treated as old, second as new), which reached F1 0.923 on templated v4. **Caveat (cross-reference to 4.3):** this is a position assumption — the same one the order-shuffle experiment breaks. It works exactly when line order is guaranteed by the harness; in any store where order is not a contract, the provenance grouping below is the correct mode, not the positional fallback. When metadata is available (`anchor_old`, `role_old`, ...), the module groups context lines by their anchor affiliation, ignoring line order entirely — which is what real memory systems can supply from their write ledger.

5. The decomposition (the shared result)

The fixed line order was a stand-in for something every real memory store already has: provenance metadata — which record set which value, and in what sequence. Restore that as an explicit ledger and the method holds with order destroyed. The task factorizes into two independent subproblems:

- **Reference resolution** (a text problem): match the candidate to the context element it refers to. This is where structural similarity — the direct contextual comparison — does the work.
- **Recency attribution** (a ledger problem): decide whether that referent set the old value or the current one. This is not a text problem at all; in a real store it is a lookup against the supersession ledger.

Neither half solves the task alone. A pure text method has to smuggle in provenance (via line order, which is why the shuffle breaks it); a pure ledger method has nothing to attribute until the reference resolves. The structural detector and the ledger turn out to be the two halves of one detector.

The decomposition is shipped on both sides. In mnemo it is a store method, `classify_reversion` (0.7.14): it embeds the candidate, scores it against the ledger's own superseded-versus-current split as a margin, attributes old/new from the supersession ledger, and abstains when the reference does not discriminate (referenced-revert 24/24, affirm-current 22/24 with two safe abstains, bare "go back" abstained 23/24, unrelated abstained 24/24 — 93/96 on a mnemo-native task). It classifies

only and never restores; a flagged revert is a signal an authorized caller acts on through the revert channel. In TAT it is TAT-Monitor's provenance mode (Section 7).

6. The boundary

The decomposition has a clean edge. Every residual false positive, on both implementations and different embedders, is a candidate whose target role appears in neither context role line: an unresolvable reference. There the correct behaviour is abstention — a confidence threshold on the reference match gives it — not a guess. And the truly value-obscuring twin, a bare "go back" with no reference at all, stays undecidable from text: no classifier can separate a stale echo from a deliberate reaffirm when the two are byte-identical, differing only in provenance. That case needs an authorization channel at the write path, not smarter reading — the direction mnemo's authorized-revert work took separately.

7. Real-noise stress test

To move past a synthetic register entirely, we built the real-noise set described in Section 2: 100 rows, balanced. Positives are half natural reverts and half keyword-free anchored ones that defer to the old setter by name; negatives include keyword-bearing distractors (revert vocabulary, keep decision) and name-bearing distractors (old setter mentioned, current value kept), so neither lexeme nor mention separates. Order is shuffled per row. A full-family shortcut audit confirms no trivial rule beats the all-positive baseline of 0.675: keyword 0.602, position 0.600, the cosine family 0.612 to 0.675, name-token 0.627, value-token and recency at 0.000. The intended signal is present (anchored positives resolve 26/26 by reference) but no shortcut extracts it past the distractors.

TAT-Monitor (the universal module packaging the cascade v1–v4, native all-MiniLM-L6-v2) on the set:

configuration	Accuracy	F1	key subset behaviour
TAT-Monitor, positional v4 (no provenance)	0.70	0.722	revert_natural 25/25; revert_anchored 14/26; keep keyword-distractors 12/12 false positives
TAT-Monitor, provenance grouping	0.88	0.895	revert_anchored 26/26 ; keep + name-distractor false positives → 0

(The v4nat AUROC of 0.964 in Section 4.2 belongs to the synthetic heldout; the stress-set numbers here are a different dataset and are reported separately on purpose.)

Per subset, the no-provenance run is exactly the operating envelope the set was built to draw: where a revert lexeme is present the keyword stage nails it (25/25); keyword-free by-role reverts fall to the positional v4 comparison and land near chance (14/26); the asserted-value path is clean (0/12 false positives on named_new); and every keep whose wording contains a revert word trips the keyword stage (12/12). Swapping only the old/new grouping for provenance — which line names the anchor that set the old value, read from the ledger — and leaving the rest of the cascade untouched lifts anchored recall to 26/26, drops the keep and name-distractor false positives to zero, and raises overall F1 from 0.722 to 0.895. The only residual false positives are the keyword distractors, produced by the v1 keyword stage before v4 runs — they belong to the keyword boundary, not the reference-resolution one.

This is the paper's thesis measured end to end on a third-party detector: reference resolution is a text problem the structural method solves; old-versus-new is provenance; and the two glued together beat either alone.

8. Generalisation to novel phrasings (held-out experiments)

To check whether accuracy was due to memorisation of template phrases: the first held-out set (6 new phrases) dropped a model trained only on the original three templates to F1 0.565 (AUROC 0.565) — surface forms had been memorised. After retraining on a mix of the original and half the held-out examples (210 total) and testing on the remaining 70 unseen examples, F1 rose to 0.744 (AUROC 0.862), with revert recall 0.94 and precision 1.0; the residual weakness sat on the keep class, indicating threshold calibration rather than a missing signal. The method can extract the structural invariant given sufficient calibration diversity, but is sensitive to template memorisation when the template count is small. (Artifacts: TAT-ROOT.)

On the full naturalized v4nat set (all 104 rows, both registers — legitimate for a training-free method, though only the 46-row heldout split carries the no-seen-phrasing guarantee), TAT-Monitor reaches Accuracy 0.933 · Precision 0.880 · Recall 0.978 · **F1 0.926** · AUROC 0.986 · confusion [tn 53, fp 6; fn 1, tp 44].

A note on the two heldout numbers reported in this paper: 0.905 and 0.930 are the SAME method on the SAME 46-row heldout split, under two different embedders — TAT-Monitor's native all-MiniLM-L6-v2 (0.905) and Agora's independent reimplement on nomic-embed-text (0.930). The gap is embedder choice, not a fixture version; that the result holds across two embedders and two implementations is itself part of the evidence.

9. Conclusion and applicability boundaries

On these fixtures the value-obscuring reversion task is well-predicted by a combination of simple structural comparison and metadata: F1 0.905 on clean linguistic data (0.930 in independent reproduction, n=46 heldout) and F1 0.895 on noisy real data with provenance (n=100), against value-match $\approx$ 0.03 and value-cosine $\approx$ 0.55. The sample sizes are small and the fixtures author-built (Section 2); the claim is scoped accordingly. The boundaries are clearly defined:

- **Keyword distractors** — phrases carrying revert lexicon without being reverts — remain the cascade's false-positive source; they belong to the keyword stage's boundary.
- **Unresolvable reference** — if the role is absent from the context, the method cannot decide; abstention is the appropriate behaviour, and a reference-match confidence threshold provides it.
- **The bare echo** — a re-assertion with no reference at all is undecidable from text alone and needs a write-path authorization channel, not a better classifier.
- **Template memorisation** — generalisation to novel phrasings requires calibration diversity; a detector trained on few templates memorises surfaces.

The main conclusion: the structural signal lives in roles, not keywords — and it becomes a *complete* detector only when glued to provenance. A task can look unsolvable to text methods, be solved by a structural method that appears to cheat, and on inspection factorize into a text half and a provenance half with a precisely bounded undecidable core.

10. Reproducibility

Every number above is re-runnable. TAT's notebooks, CSVs, metrics and the TAT-Monitor module are in TAT-ROOT (`experiments/value_obscuring_rastislav`) and TAT-ONE-TAP (`tat_monitor/`); Agora's fixtures, generators, and audit probes are in the agora repository (`v4nat_decomposition_probe.py`, `build_realnoise_stress_v1.py`, `tat_monitor_runner.py`, and the fixtures under `agora_output/public_fixtures/`). The audit lessons are stated plainly, including the two fixture shortcuts we shipped and retracted, the cosine-baseline claim published too broadly and corrected, and the memorised template result reported alongside the fix.

References

- Alchourrón, C. E., Gärdenfors, P., & Makinson, D. (1985). On the logic of theory change: Partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2).
- Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, 12(3).
- Snodgrass, R. T. (1999). *Developing Time-Oriented Database Applications in SQL* (bitemporal data models).
- Cohen, R., Biran, E., Yoran, O., Globerson, A., & Geva, M. (2024). Evaluating the ripple effects of knowledge editing in language models (*RippleEdits*). *TACL*.
- Data, fixtures, and audit probes: github.com/DanceNitra/agora (`agora_output/public_fixtures/`, `mnemo/probes/`). TAT-Monitor and experiment notebooks: TAT-ROOT / TAT-ONE-TAP.

Acknowledgments

The authors thank members of the Cophy Runtime and HeartFlow communities for valuable discussions and cross-framework validation.

Merged draft v1 (2026-07-12) — both authors review before publication. The public version passes Agora's full validate → storm → audit → verify gate before it ships.